A3 with generative AI: building your process capability notebook in Claude

You may use a generative AI assistant to help build your A3 notebook. This guide shows one way to do it with a free Claude account at claude.ai. It takes about three hours, most of it checking what Claude gives you. Claude writes and runs the Python quickly, and it can hand you a Jupyter notebook with its outputs already saved. The marks for choosing the baseline window, reading the capability indices, setting the alarm threshold and advising the quality manager come from your own judgement about the Port Louis liquid-fill line, so the guide keeps those parts in your hands.

Before you start

Sign in at claude.ai with a free account. Open Settings, then Capabilities, and turn on **Cloud code execution and file creation**. Claude needs it to read your CSVs, run the notebook and hand you a real `.ipynb` file. You do not need to add any other skill.

You also need a Google account for Google Colab, where you will run the finished notebook yourself.

If Claude stops and asks permission to run a command, click **Allow once**. The command runs in Claude's own sandbox in the cloud, not on your computer. A prompt that builds a file can take 5 to 15 minutes. Leave the tab open until Claude has finished and the file appears in the chat.

Start a new chat for this assignment and keep the whole job in that one chat, so Claude remembers what it has already built. A free account has a usage limit that resets every five hours. If you hit it, wait and carry on in the same chat.

Files to upload

Download these from the A3 assignment page on InnoSkills and attach all five to your first message, using the paperclip button:

1. `A03_process_capability_jupyter_analysis.pdf`, your brief.
2. `A03_port_louis_pasteurisation_temp_10000.csv`, the 10,000 one-second temperature readings.
3. `A03_spirulina_fill_volume_200x5.csv`, the 200 fill-volume subgroups.
4. `A03_exemplar_dar_es_salaam_hibiscus_process_capability.ipynb`, the worked exemplar notebook.
5. `A03_exemplar_dar_es_salaam_hibiscus.pdf`, the marker's note on the exemplar.

The exemplar is set at a different plant (a daily hibiscus yield record in Dar es Salaam, with a one-sided floor), so its charts, indices and detectors answer questions your data does not ask. Use it to see the shape of a strong notebook: the data checked before it is trusted, limits worked from the constants, two fits compared with a stated choice, thresholds tuned against evidence, and markdown between every code cell.

Prompts

Copy each prompt into the chat in turn. Read the answer before you send the next one. Where a prompt has a square-bracketed gap, fill it in yourself.

1. Read the brief

```
I'm a learner on the DigiSkills programme. Attached are my A3 brief (PDF), its two datasets (CSV), and a worked exemplar notebook plus its marker's note from a different plant. Read the brief carefully. Then give me a checklist of every Key Requirement and every row of the marking scheme, with what a Distinction answer needs for each. Do not build anything yet. My submission is about the Port Louis Spirulina Power liquid-fill line, not the exemplar's Dar es Salaam hibiscus plant.
```

2. Check the data

```
Load both CSVs with code, skipping the comment lines at the top. For each, tell me: the column names and types, the specification limits and target stated in the header, the number of rows, and any missing values, duplicates, gaps in the timestamps or values that look like errors. Then plot the temperature series over time and describe where it is steady and where it starts to move. Do not clean anything and do not choose a baseline window; just report what you find.
```

Decide how to handle each data problem and which stretch of the temperature run to fit your limits on. Both decisions are part of your mark.

3. Start the notebook

```
Create a Jupyter notebook named A03_Process_Capability_[YourSurname].ipynb. It must read each CSV by its bare filename, so it runs in Google Colab once the files are uploaded beside it. Sections, each with a markdown heading: setup (imports and a fixed random seed), loading and checks (my cleaning decisions: [describe them]), exploratory analysis for both datasets (descriptive statistics including skewness and kurtosis, histograms, box plots, time series, multi-panel and labelled), and normality testing (Shapiro-Wilk or Anderson-Darling plus Q-Q plots for both). Put a short markdown cell after each output stating what it shows, with the numbers. Follow the exemplar's structure but none of its code comments, numbers or wording.
```

4. Add the control charts

```
Add a control chart section. Write the formulas yourself from the constants; do not use an SPC library. Build X-bar and R charts for fill volume (n = 5) and an I-MR chart for temperature, with 1, 2 and 3 sigma zones and out-of-control points marked. Fit the temperature limits twice: once on readings [first and last reading of your baseline window], applied across the whole run, and once on all 10,000 readings. Show both on one figure and print a table comparing the two sets of limits and the points each one flags. Leave a headed empty markdown cell for my choice.
```

5. Add the capability analysis

```
Add a capability section for fill volume: Cp, Cpk, Pp and Ppk against the limits in the file header, with 95% confidence intervals, the expected underfill and overfill in parts per million, and the count actually outside the limits in the data. Show a histogram with both limits and the within and overall normal curves. Leave a headed empty markdown cell for my interpretation.
```

6. Add the anomaly detector

```
Add an anomaly detection section for temperature. Implement a rolling z-score with a 60-second window and a CUSUM. Tune each one's threshold on my baseline window so that it raises no more than [your false-alarm budget, for example no more than two false alarms in the baseline window], with the rate per hour estimated by simulation, and print a table of threshold against false alarms. Then mark the anomalies each detector finds on the full time series and report when each first alarms. Leave a headed empty markdown cell for my choice of detector and threshold.
```

7. Run it and check the numbers independently

```
Execute the whole notebook top to bottom so the saved file carries its outputs, and tell me about any cell that fails. Then, in a separate script that does not read the notebook, recompute the grand mean, R-bar, the X-bar, R and I-MR limits and the four capability indices from the CSVs. Show me a table comparing those with the notebook's printed values and flag any difference. Give me the executed .ipynb to download.
```

If anything disagrees, ask Claude to find and fix the cause, then run this prompt again.

8. Run it yourself in Google Colab

Open colab.research.google.com and choose File, then Upload notebook, and pick your `.ipynb`. Open the Files panel on the left and upload both CSVs. Choose Runtime, then Run all. Every cell must finish without an error and the charts must match the ones Claude showed you. If a cell fails, paste the error into the Claude chat and ask for a fix, then run all again.

9. Write the judgement cells yourself, then get feedback

Write the four empty markdown cells in your own words: which temperature limits you kept and why, what the capability indices mean for underfilled and overfilled bottles leaving Port Louis, which detector and threshold you would run, and the recommendations for the quality manager. Then paste each draft into the chat:

```
Here is my draft [baseline choice / capability interpretation / detector choice / recommendations]. Mark it against the "[Control Chart Construction / Capability Analysis / Anomaly Detection / Code Quality and Narrative]" row of the marking scheme in my brief. Tell me which band it reaches and what would lift it one band. Point out any figure in it that does not match the notebook. Do not rewrite it for me. My draft: [paste your draft here]
```

When you are happy with a draft, paste it into its cell in Colab yourself.

Check before you submit

- In Colab, choose Runtime, then Restart and run all, and confirm every cell runs from a clean start.
- Work out one control limit by hand from R-bar and the A2 constant, and check it matches the chart.
- Look at every chart. Each needs labelled axes, a centre line, both limits and the flagged points marked.
- Confirm there is a markdown cell between the code cells explaining your reasoning, not just output.

- Read every cell for leftover exemplar content: Dar es Salaam, hibiscus, calyx, hall temperature or shift return should not appear anywhere.
- Download the notebook from Colab with File, then Download, then Download .ipynb, and name it `A03_Process_Capability_[YourSurname].ipynb` with your own surname.

Add a short markdown cell at the top saying which parts Claude built and which decisions and writing are yours. Claude can make mistakes. You are responsible for every number and every sentence in what you submit.